

CENTRE FOR INTERNET OF THINGS

Planning and Discussion for Minor Project

“IoT-Powered Emergency Alert and Disaster Monitoring System”

Submitted by
Mohd Faisal Khan(0901IO231035)

Submitted to
Smita Maithil
Assistant Professor



Centre for Internet of Things
Madhav Institute of Technology & Science, Gwalior
Gole ka Mandir, Gwalior - 474005, M.P., India

1. Project Roadmap Overview

The development of the Disaster Monitoring and Early Warning System follows a structured roadmap consisting of requirement analysis, system design, development, testing, and deployment. The roadmap is designed to ensure a systematic progression from concept to implementation while maintaining scalability and reliability. Since the project is entirely software-based and dependent on API data, emphasis is placed on efficient data handling, real-time processing, and user-friendly visualization.

The roadmap ensures that each phase contributes to building a robust system capable of monitoring disasters such as earthquakes and extreme weather conditions using real-time data sources.

2. Requirement Analysis

In this phase, the functional and non-functional requirements of the system are identified and documented. The primary objective is to define what the system should do and how it should perform under various conditions.

- The system must fetch real-time data from external APIs such as weather and earthquake services.
- It should process and analyze incoming data to detect abnormal or hazardous conditions.
- The system must generate alerts based on predefined thresholds or logic.
- A user interface should be provided to display real-time data and alerts clearly.
- Non-functional requirements include system reliability, scalability, low latency, and ease of use.

This phase establishes the foundation for system development and ensures alignment with project objectives.

3. System Design

The system design phase focuses on defining the architecture, components, and data flow. Since the project is software-only, the architecture is based on a layered model.

- The frontend layer is responsible for displaying data and alerts through dashboards.
- The backend layer handles data processing, business logic, and API integration.
- The API integration layer connects the system with external data sources such as weather and seismic APIs.

The system follows a modular design approach, allowing easy addition of new disaster types or APIs in the future. Data flow is designed to ensure real-time updates and minimal delay in alert generation.

4. Technology Selection

Appropriate technologies are selected to ensure efficient implementation and performance.

- Frontend technologies include HTML, CSS, and JavaScript, with optional frameworks like React for enhanced user experience.
- Backend technologies include Node.js or Python (Flask/Django) for handling server-side logic.
- APIs such as OpenWeatherMap and USGS Earthquake API are used for real-time data retrieval.
- Optional databases like MongoDB or MySQL can be used to store historical data for analysis.

The selection of technologies is based on ease of integration, scalability, and real-time processing capability.

5. Data Collection and Integration

This phase involves integrating external APIs and fetching real-time data.

- API endpoints are configured to retrieve data in JSON format.
- The system periodically requests updated data using scheduled calls or polling mechanisms.

- Data such as temperature, rainfall, wind speed, and earthquake magnitude is extracted and stored temporarily for processing.

Proper error handling is implemented to manage API failures or network issues, ensuring system stability.

6. Data Processing and Risk Analysis

In this phase, the collected data is analyzed to identify potential disaster conditions.

- The system applies predefined thresholds to determine risk levels.
- For example, an earthquake magnitude above a certain value triggers a high-risk alert.
- Weather parameters such as heavy rainfall or strong winds are evaluated to detect possible floods or storms.

The processing logic is designed to be efficient and fast, enabling real-time decision-making.

7.Alert Generation and Notification

Once a risk is detected, the system generates alerts for users.

- Alerts are categorized into different levels such as low, medium, and high risk.
- Notifications are displayed on the user interface in real time.
- Optional features include sending alerts via email or SMS for wider dissemination.

The goal is to ensure that users receive timely and accurate warnings to take preventive action.

8. User Interface and Visualization

The system includes a user-friendly interface for monitoring and analysis.

- A dashboard displays real-time data in the form of charts, graphs, and maps.
- Alerts are highlighted prominently for quick identification.
- Users can select locations and view disaster-related data specific to that region.

Visualization enhances understanding and helps users interpret data effectively.

9. Testing and Validation

Testing ensures that the system functions correctly under various conditions.

- Unit testing is performed for individual modules such as API integration and data processing.
- Integration testing ensures smooth communication between frontend and backend components.
- System testing evaluates overall performance, including real-time data handling and alert accuracy.

Validation is done using sample data and real-world scenarios to ensure reliability.